

第 34 讲

SGD 与 Mini-Batch SGD

为什么训练通常不用整个数据集一次算一个 Gradient

《面向 LLM / Agent Systems 科研的深度学习零基础课》

【本讲定位】第 33 讲我们把 Gradient Descent 拆到了最底层：先用全部数据计算一个 Gradient，再沿负梯度方向更新参数。本讲只改变一件事：每次更新时到底用多少训练样本来估计 Gradient？这会自然得到 Full-Batch、SGD 与 Mini-Batch SGD，并直接连接 PyTorch 的 DataLoader、LLM 的 micro-batch / effective batch，以及以后实验中经常看到的 batch size 超参数。

学习目标

学完本讲，你应该能够：

1. 严格区分 Full-Batch Gradient Descent、SGD、Mini-Batch SGD；
2. 解释为什么每个 Mini-Batch 算出的 Gradient 会波动；
3. 理解 Mini-Batch Gradient 是 Full-Batch Gradient 的随机估计；
4. 解释 Batch Size 改变后，Gradient Noise、Step 数和内存/吞吐量怎样变化；
5. 正确计算一个 Epoch 有多少个 Optimizer Step；
6. 理解 torch.optim.SGD 与 DataLoader 的 batch_size 不是同一层概念；
7. 理解 Gradient Accumulation 与 Effective Batch Size；
8. 知道为什么改变 Batch Size 后 Learning Rate 往往需要重新验证；
9. 看懂论文 Training Setup 中的 batch size / global batch / accumulation；
10. 避免用“相同 Epoch”或“最终 Loss”做不公平比较。

1 先回到第 33 讲：Full-Batch Gradient Descent 做了什么？

假设训练集有 N 个样本，每个样本产生一个损失：

$$\ell_i(\theta)$$

训练集平均 Loss 可以写成：

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_i(\theta)$$

那么 Full-Batch Gradient 就是：

$$g_{full} = \nabla L(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla \ell_i(\theta)$$

也就是说：把全部训练样本都看一遍，算出一个平均 Gradient，然后参数只更新一次。

假设：

$$N = 64$$

那么 Full-Batch 的一个 Epoch 是：

$$64 \text{ 个样本} \rightarrow 1 \text{ 个 Gradient} \rightarrow 1 \text{ 次 optimizer.step()}$$

Full-Batch 的 Gradient 对当前训练集而言最“完整”：同一组参数、同一训练集下，每次计算得到同一个 Gradient（暂不考虑 Dropout 等随机层）。但代价是每更新一次参数，都要先处理完整数据集。

2 为什么不总用 Full-Batch?

如果训练集只有几十个样本，Full-Batch 并不难。但真实深度学习中，数据集可能很大，而且单个样本本身也可能非常重。

以 LLM 为例，一个训练样本可以包含数千个 Token。模型 Forward 时还需要保存大量中间激活，以供 Backward 使用。因此如果试图让整个训练集一次性进入模型：

- GPU / RAM 通常放不下；
- 每次 Parameter Update 要等待完整数据集处理完；
- 无法方便地利用有限显存做持续更新；
- 实际训练吞吐与并行方式也不一定合理。

于是核心思想变成：

我不必每次都使用全部样本精确计算 Full Gradient；能否随机拿一小部分样本，先估计一个大致方向，然后先走一步？

这就是 Stochastic / Mini-Batch 方法的来源。

3 三个概念必须严格区分

假设训练集共有 N 个样本，每次更新使用 B 个样本。

方法	每 Step 样本数	B	直觉
Full-Batch GD	全部 N 个	$B = N$	每次用完整训练集算 Gradient
SGD (严格意义)	1 个	$B = 1$	每看一个随机样本就更新一次
Mini-Batch SGD	一小批	$1 < B < N$	每批算平均 Gradient，再更新一次

术语存在工程习惯差异。现代深度学习代码中，大家经常把“使用 Mini-Batch + SGD 更新规则”的训练也简称为 SGD。例如 `torch.optim.SGD` 并不意味着 batch size 必须等于 1。Optimizer 只管“拿到梯度后如何更新参数”；究竟每次用几个样本，是 DataLoader / 数据管线决定的。

4 Mini-Batch Gradient 的数学形式

设本轮随机抽到的 Mini-Batch 为：

$$\mathcal{B} = \{i_1, i_2, \dots, i_B\}$$

那么该 Batch 的平均 Loss：

$$L_{\mathcal{B}}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \ell_i(\theta)$$

对应 Gradient：

$$g_{\mathcal{B}} = \nabla L_{\mathcal{B}}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i(\theta)$$

参数更新仍然是：

$$\theta_{t+1} = \theta_t - \eta g_{\mathcal{B}}$$

注意，和第 33 讲相比，更新公式没有变。变化的是 Gradient 的来源：

$$\nabla L_{full} \longrightarrow g_{batch}$$

5 【具体数字例子】为什么不同 Batch 会产生不同 Gradient？

为了先把直觉讲透，假设只有一个参数 w ，4 个训练样本在当前位置分别给出 Gradient：

$$[2, 4, 6, 8]$$

Full-Batch Gradient 为：

$$g_{full} = \frac{2 + 4 + 6 + 8}{4} = 5$$

如果 Batch Size=2，随机抽到样本 1 和 4：

$$g_{batch} = \frac{2 + 8}{2} = 5$$

这次刚好等于 Full Gradient。

但如果抽到样本 1 和 2：

$$g_{batch} = \frac{2 + 4}{2} = 3$$

抽到样本 3 和 4：

$$g_{batch} = \frac{6 + 8}{2} = 7$$

因此每一个 Mini-Batch 都像是从完整数据中拍了一张“小照片”：大体反映整体方向，但会有随机偏差。

如果 Full Gradient 是“全班 64 人平均身高”，Mini-Batch Gradient 就像随机抽 8 个人估计全班平均身高。抽样越少，估计值通常越抖；抽样越多，通常越接近全班真实平均。

6 Gradient Noise 到底是什么？

把 Full Gradient 记为：

$$g = \nabla L(\theta)$$

把某个随机 Mini-Batch Gradient 记为：

$$\hat{g}$$

可以把两者差异直观写成：

$$\hat{g} = g + \epsilon$$

其中 ϵ 就可以理解为本轮抽样带来的 Gradient Noise。

对于均匀随机抽样，在常见条件下，Mini-Batch Gradient 的期望会围绕 Full Gradient：

$$\mathbb{E}[\hat{g}] \approx g$$

直觉上就是说：

单独一次可能偏左、偏右，但如果不断随机抽许多 Batch，平均起来会靠近完整数据给出的方向。

对于近似独立的样本平均，Batch Size 增大时估计方差通常下降，常见直觉尺度是：

$$\text{Var}(\hat{g}) \propto \frac{1}{B}$$

但真实训练里样本并非总是独立、还可能无放回采样，因此不要把 $1/B$ 当成任何数据集都严格成立的万能定律。

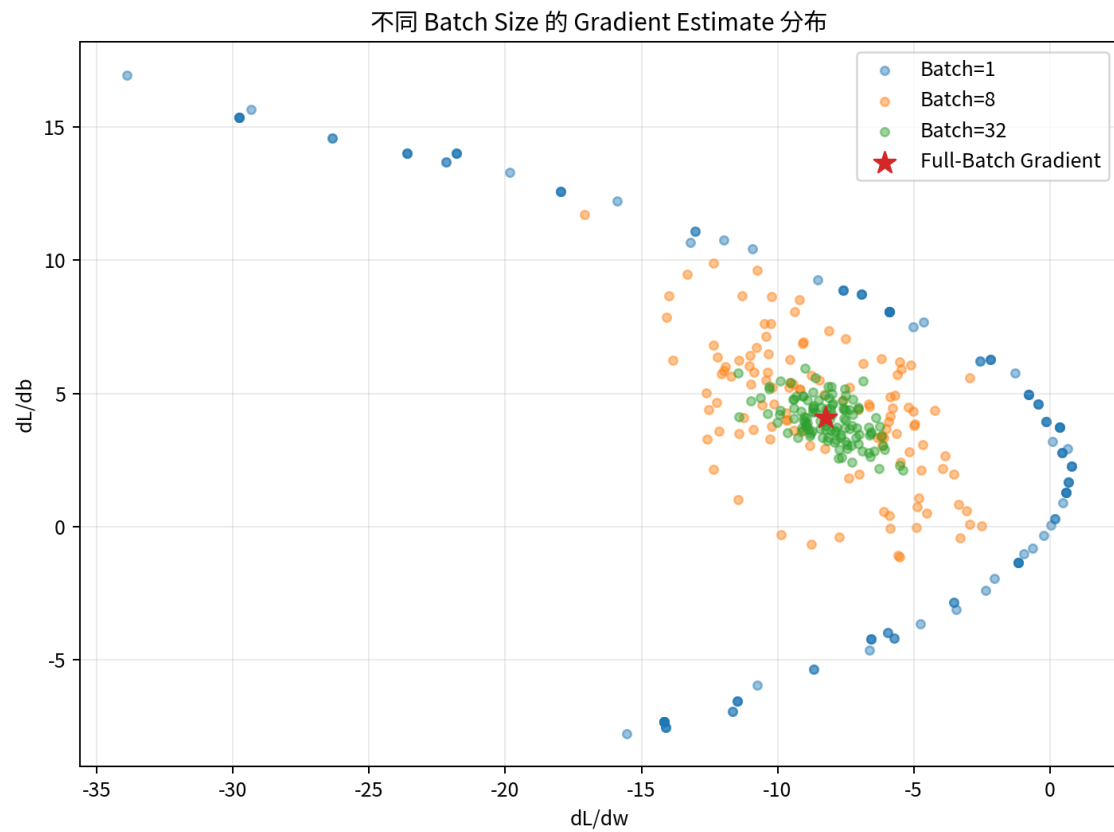


图 1: 同一参数点上, 不同 Batch Size 得到的 Gradient Estimate 分布。星号是 Full-Batch Gradient。

实验中, 在同一组参数上:

Batch Size	$\text{std}(dL/dw)$	$\text{std}(dL/db)$	平均 Gradient	误差范数
1	8.8183	7.1767		10.2810
4	4.2362	3.4047		4.7569
8	2.9476	2.4251		3.4060
16	2.0359	1.4983		2.2290
32	1.1442	0.8875		1.2611
64	0	0		0

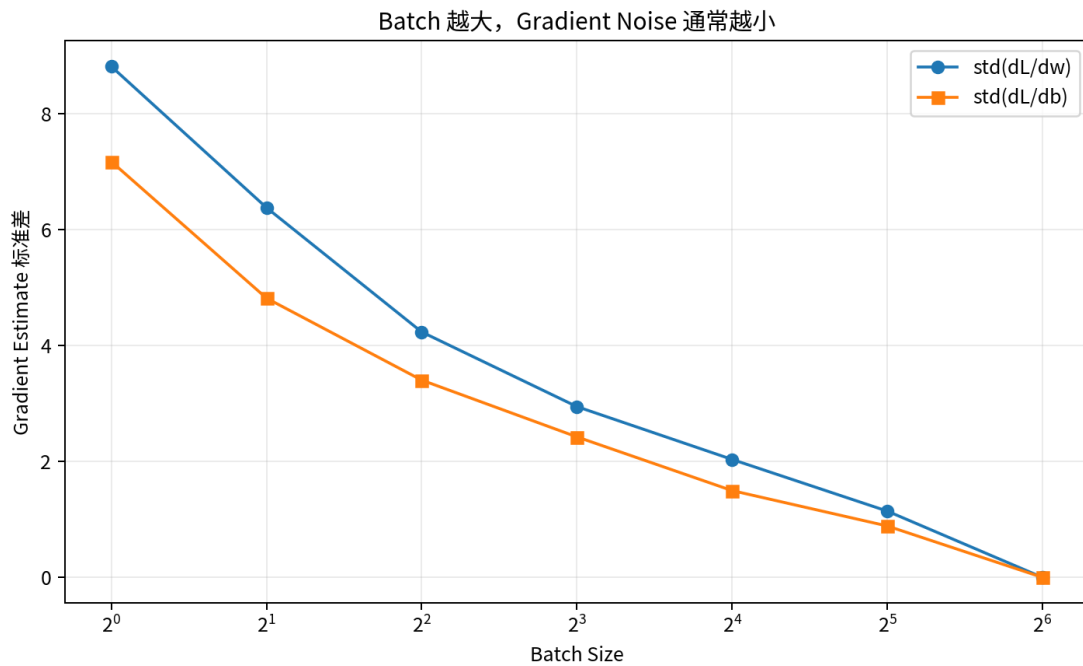


图 2: 在本讲固定实验中, Batch Size 增大时 Gradient Estimate 的标准差明显下降。

7 Gradient Noise 是 Bug 吗?

不是。它首先是“只看部分数据”自然产生的统计波动。

它既可能带来问题, 也可能带来某些有利效果:

- 波动过大: Loss 曲线抖动、更新方向不稳定、可能需要更谨慎的 Learning Rate;
- 适度随机性: 不会每一步都严格沿完整训练集的确切方向, 有时有助于优化过程探索不同区域;
- 但不能机械地说“噪声越大泛化越好”或“小 Batch 一定更好”。

论文如果声称某 Batch Size 带来更好泛化, 至少要检查: Learning Rate 是否重新调过、总 Step / 总 Token 是否一致、训练预算是否一致、随机种子是否重复、模型是否真正训练到可比较状态。否则 Batch Size 与训练预算可能混在一起。

8 Batch Size 会直接改变一个 Epoch 有多少 Step

假设:

$$N = 64$$

如果:

$$B = 8$$

那么:

$$\text{Steps per Epoch} = \frac{64}{8} = 8$$

如果:

$$B = 32$$

则：

$$\text{Steps per Epoch} = 2$$

如果 N 不能整除 B ，且 `drop_last=False`：

$$\text{Steps per Epoch} = \left\lceil \frac{N}{B} \right\rceil$$

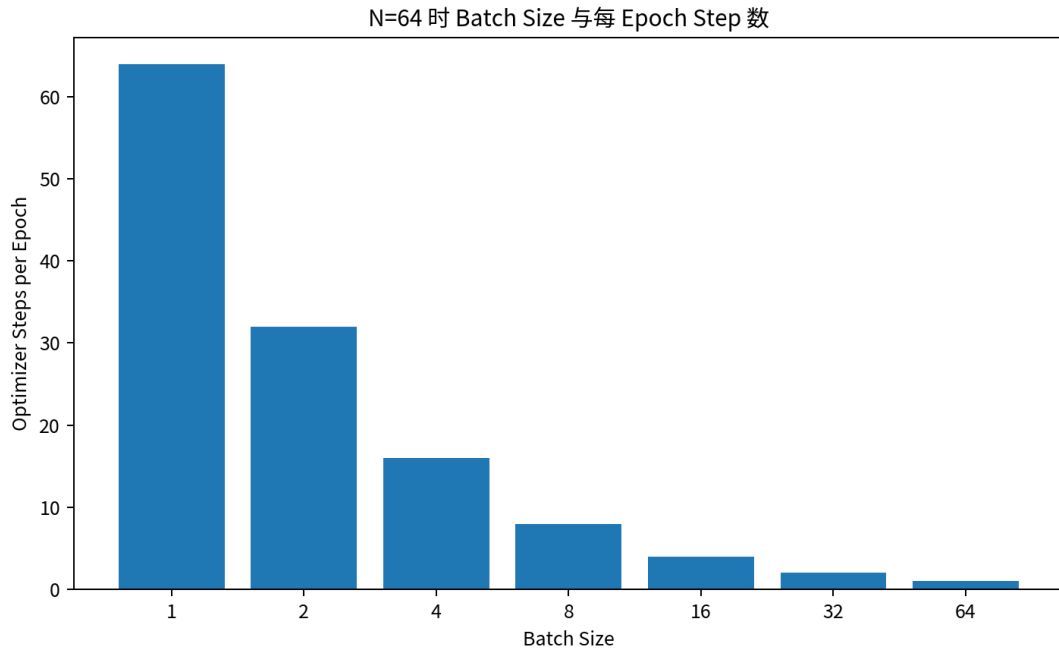


图 3: 同一数据集 $N = 64$ 时，Batch Size 越大，每个 Epoch 的 Optimizer Step 数越少。

“训练 30 Epoch” 并不表示不同 Batch Size 做了同样数量的 Parameter Update。Batch=1 时每 Epoch 64 次 Step；Batch=64 时每 Epoch 只有 1 次 Step。

9 为什么比较 Batch Size 时经常会出现“苹果和橘子”问题？

假设比较：

$$B = 1 \quad \text{vs} \quad B = 64$$

都训练 30 Epoch。

那么：

$$B = 1 : 30 \times 64 = 1920 \text{ Steps}$$

而：

$$B = 64 : 30 \times 1 = 30 \text{ Steps}$$

两者虽然看过的样本总数相同，但参数更新次数差了 64 倍。

因此实验至少要明确比较基准：

1. 相同 Epoch：看过相同轮数的数据；
2. 相同 Optimizer Steps：更新参数次数相同；

3. 相同 Seen Samples / Tokens：处理过的数据量相同；
4. 相同 Wall-clock / FLOPs：计算预算尽量接近。

这些问题没有一个永远正确的标准，取决于 RQ。但必须写清楚。

10 【实验】相同 Epoch 数下不同 Batch Size 会怎样？

本讲用同一个数据集：

$$y = 3x - 2 + noise$$

固定相同初始化、相同基本 Learning Rate，分别训练：

$$B \in \{1, 8, 32, 64\}$$

每个都训练 30 Epoch。

结果：

B	Step/Epoch	Total Step	Final Epoch Loss	Final (w, b)
1	64	1920	0.0527	(2.993,-2.052)
8	8	240	0.0422	(2.976,-2.044)
32	2	60	0.0412	(2.987,-2.042)
64	1	30	0.0592	(2.943,-1.939)

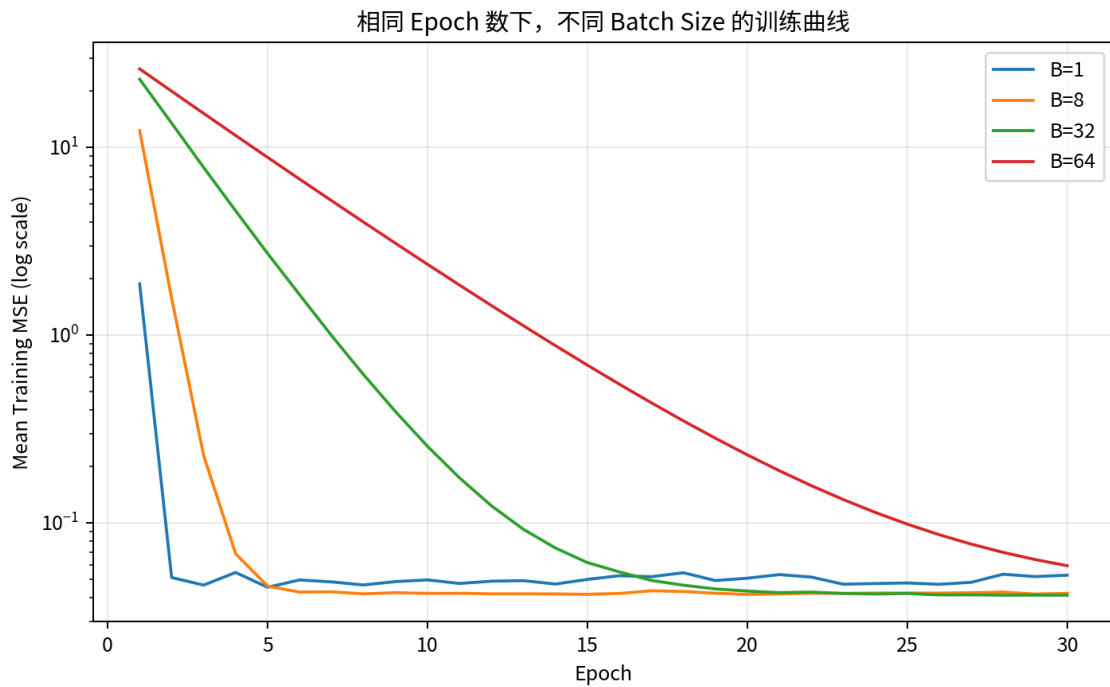


图 4: 相同 Epoch 数下，不同 Batch Size 的训练 Loss。注意：对应的 Optimizer Step 数并不相同。

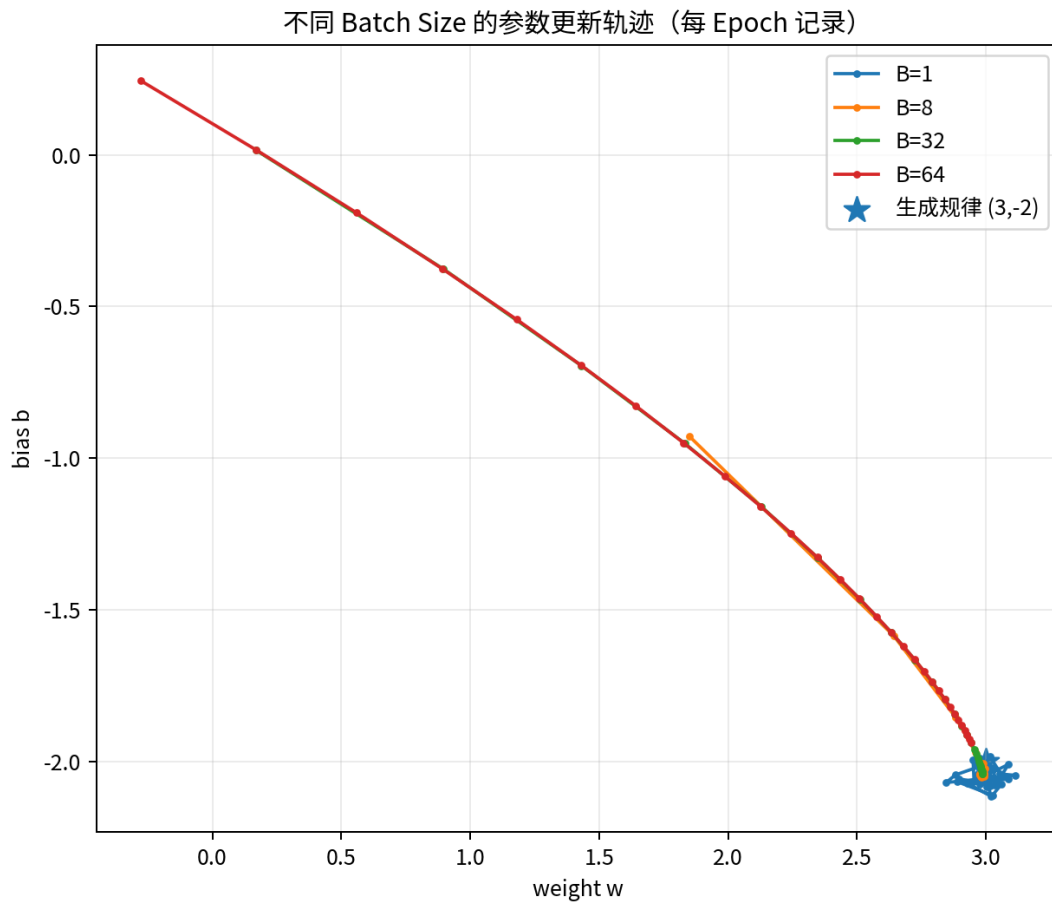


图 5: 参数 (w, b) 的训练轨迹。不同 Batch Size 会带来不同的更新频率与噪声。

这个玩具实验不能推出“ $B=32$ 最优”。它只说明 Batch Size 改变以后, Gradient Noise、Step 数和参数轨迹都会改变。真实任务必须重新做控制实验。

11 为什么 Mini-Batch 成为常见折中?

Mini-Batch 同时兼顾四个现实问题:

1. **内存**: 不需要一次把全部训练集送进模型;
2. **向量化**: 比单样本 SGD 更容易利用 GPU 的并行计算;
3. **更新频率**: 比 Full-Batch 更频繁地更新参数;
4. **噪声**: 通常比单样本 SGD 稳定。

因此现代深度学习最常见的不是严格 $B = 1$ 的 SGD, 也不是完整 Full-Batch, 而是:

Mini-Batch + Optimizer

Optimizer 可以是 SGD、Adam、AdamW 等。

12 PyTorch 中：谁决定 Batch？谁负责 SGD？

下面两行属于不同层次：

```
loader = DataLoader(dataset, batch_size=32, shuffle=True)
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

第一行决定：

每个 Training Step 给模型多少条样本。

第二行决定：

Backward 得到 Gradient 后，参数按什么更新规则改变。

典型 Training Loop：

```
for xb, yb in loader:
    optimizer.zero_grad()      # 清除旧梯度
    pred = model(xb)           # 当前 mini-batch forward
    loss = loss_fn(pred, yb)    # 当前 mini-batch mean loss
    loss.backward()             # 当前 mini-batch gradient estimate
    optimizer.step()            # 使用这份 gradient 更新参数
```

因此“Mini-Batch SGD”不是某个神秘 API。它只是：DataLoader 每次给一小批数据，Backward 算这批数据的 Gradient，Optimizer 用它更新一次参数。

13 Batch Size 改变后，Learning Rate 为什么不能想当然地保持不变？

Batch Size 改变会影响：

- Gradient Estimate 的方差；
- 每 Epoch 更新次数；
- 每次更新看过的数据量；
- 硬件吞吐和数值行为。

因此一个 Batch Size 上合适的 Learning Rate，不一定在另一个 Batch Size 上仍合适。

有些大规模训练实践会使用与 Batch Size 相关的 Learning Rate Scaling，但这不是“永远把 LR 按 Batch 成比例放大”的万能定律。模型、Optimizer、Warmup、训练阶段都会影响结果。

如果论文只写“我们把 batch size 从 32 改到 256，性能下降”，却不说明 Learning Rate、Step 数、Warmup、总 Token 是否重新匹配，那么很难判断下降究竟来自 Batch Size 本身，还是训练超参数没有重新适配。

14 Gradient Accumulation: 显存不够, 怎样得到更大的 Effective Batch?

假设希望一次更新相当于:

$$B_{effective} = 16$$

但显存一次只能放:

$$B_{micro} = 4$$

可以连续做 4 个 micro-batch 的 Forward/Backward, 但先不 step():

$$4 \times 4 = 16$$

然后再统一更新参数。

如果每个 micro-batch 的 Loss 使用 mean reduction, 为了让最终累积梯度匹配一个大 Batch 的 mean gradient, 典型写法是:

```
optimizer.zero_grad()

for micro_x, micro_y in four_micro_batches:
    pred = model(micro_x)
    loss = loss_fn(pred, micro_y) / 4
    loss.backward()          # gradient 累积, 但暂不 step

optimizer.step()
```

本讲实验中:

$$B = 16 \text{ 一次计算}$$

得到:

$$dw = -26.13723755, \quad db = 16.71434021$$

而:

$$4 \times B_{micro} = 4$$

正确缩放后累积得到:

$$dw = -26.13724136, \quad db = 16.71433830$$

差异只有浮点误差级别。

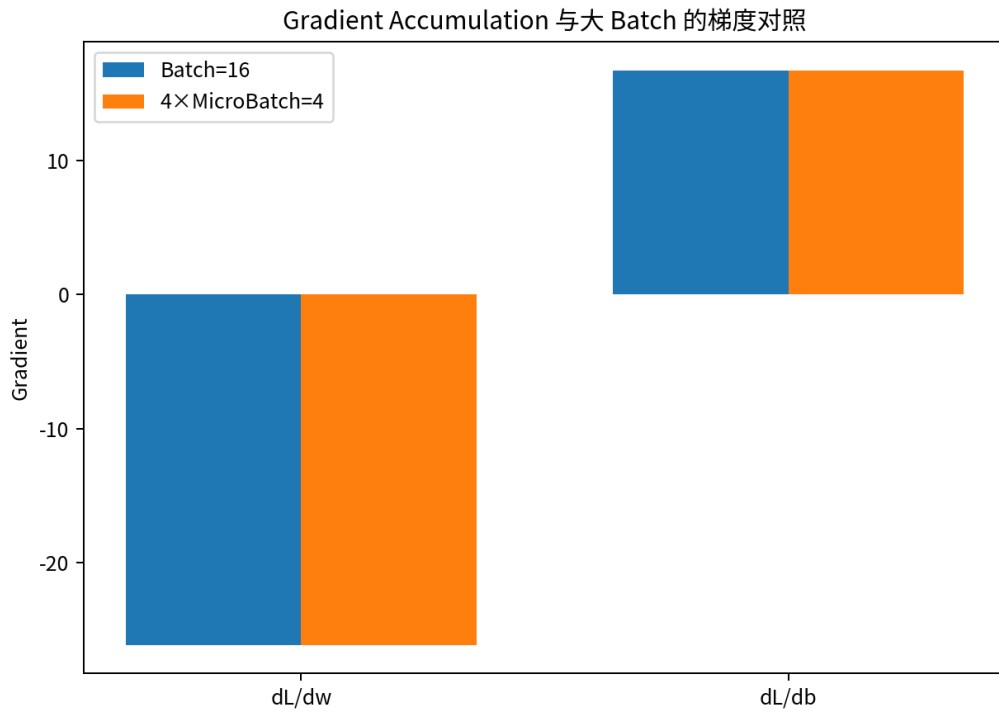


图 6: 一次 Batch=16 与 4 个 Micro-Batch=4 的累积梯度几乎一致。

15 Effective Batch Size 在多 GPU / LLM 训练中的常见形式

如果每张 GPU 每次放：

$$B_{micro}$$

共有：

$$D \text{ 张设备}$$

每更新一次前累积：

$$A \text{ 个 micro-step}$$

那么以“序列样本数”计的常见 global effective batch 近似为：

$$B_{global} = B_{micro} \times D \times A$$

例如：

$$B_{micro} = 2, \quad D = 8, \quad A = 4$$

则：

$$B_{global} = 64$$

但 LLM 中还要继续问：

- 每条序列长度是否相同？
- Padding Token 是否计入有效训练 Token？
- 论文报告的是 sequences / batch，还是 tokens / batch？

所以大模型训练中常常“每次更新处理了多少有效 Token”比单纯的样本数更有解释力。

16 Batch Size 与显存、吞吐量：不要只背“越大越快”

增大 Batch Size 常常可以让 GPU 计算更饱满，提高吞吐，但只到某个范围。继续增大可能：

- 激活显存明显增加；
- 达到显存上限；
- 通信或数据加载成为瓶颈；
- 单 Step 更重；
- 优化超参数需要重新适配。

因此工程上需要区分：

samples/sec 或 tokens/sec

和：

time-to-target-quality

前者高，不代表达到目标指标所需总时间一定最短。

17 与 LLM / Transformer 的关系

在 LLM 里，模型输入常见：

$input_ids : (B, T)$

Embedding 后：

(B, T, D)

这里的 B 就是 Batch Dimension。

一个 Mini-Batch 中所有 Token 共同产生 Loss，Backward 后得到这一批 Token 对巨大参数集合的 Gradient Estimate。随后 AdamW 等 Optimizer 才更新模型。

所以以前在 LLM 课里看到：

- micro batch size；
- global batch size；
- gradient accumulation steps；
- tokens per update；

本质上都建立在本讲“用多少数据估计一次 Gradient”这个问题上。

18 与 Agent Systems 科研的关系

Agent Systems 里一个“训练样本”未必是独立的一行表格，它可能是：

- 一个 trajectory；
- 一轮 interaction；
- 一段 tool-use trace；
- 一个 preference pair；
- 一个 episode 中切出的多个 step。

这时 Batch 的统计独立性可能比图像分类更复杂。例如同一条 trajectory 中相邻 step 强相关。如果把同一 episode 的大量片段随机散到 Train / Validation，不仅影响 Gradient 的相关性，还可能引入 Data Leakage。

以后做 Agent 数据实验时，不要只问“batch size= 多少”，还要问“Batch 内样本的独立单位是什么？同一 trajectory / task / user / environment 的样本是否高度相关？”

19 科研实验中怎样报告 Batch 相关设置？

至少建议记录：

- per-device micro batch size；
- gradient accumulation steps；
- number of devices；
- global/effective batch size；
- sequence length 或有效 tokens/update (LLM 时尤其重要)；
- optimizer 与 learning rate；
- shuffle / sampler；
- total steps / epochs / total tokens；
- 是否使用 drop_last；
- 随机种子与重复次数。

20 常见误区

误区 1: `torch.optim.SGD` 就意味着 batch size=1。错误。Optimizer 不决定 Batch Size。

误区 2: Batch 越大 Gradient 越准确, 所以越大一定越好。错误。优化、内存、吞吐、更新次数、Learning Rate、泛化都一起变化。

误区 3: 相同 Epoch 就是公平比较。未必。Batch 改变后 Step 数会改变。

误区 4: Mini-Batch Loss 抖动说明代码错了。未必。不同 Batch 的样本组成不同, 本来就会产生 Gradient / Loss 波动。

误区 5: Gradient Accumulation 后不需要缩放 Loss。若每个 micro-batch 使用 mean loss, 直接累加通常会让梯度放大 accumulation steps 倍; 要根据 reduction 和目标定义正确缩放。

误区 6: 大 Batch 与 Gradient Accumulation 在所有情况下完全等价。只有在特定条件下梯度数学上可对齐; BatchNorm、随机性、Dropout、数据增强、优化器状态更新时点等都可能造成实际差异。Transformer 常用 LayerNorm/RMSNorm, 但仍不能把“任何训练配置都完全等价”当作定理。

21 本讲最低闭环能力

学完以后, 看到下面代码:

```
loader = DataLoader(dataset, batch_size=8, shuffle=True)
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

for xb, yb in loader:
    optimizer.zero_grad()
    loss = loss_fn(model(xb), yb)
    loss.backward()
    optimizer.step()
```

你应该能自动翻译成:

每次随机取 8 个样本, 用它们估计当前参数的 Gradient; SGD Optimizer 用这份估计更新一次参数; 遍历完整个 Dataset 叫一个 Epoch。

并且当别人告诉你“Batch Size 从 8 改成 64”时, 你应该立刻继续问:

1. 每 Epoch Step 数变成多少?
2. Learning Rate 是否调整?
3. 总 Step / 总 Token 是否保持一致?
4. 显存和吞吐怎样变化?
5. Gradient Noise 怎样变化?
6. 比较结论到底是在控制什么变量后得到的?

下一讲预告

下一讲：**第 35 讲 Batch、Epoch、Step：训练日志中的三个时间尺度到底怎样换算**。虽然本讲已经使用了这三个词，下一讲会把它们系统化，并加入 Dataset Size、drop_last、Gradient Accumulation、global step 与 LLM token budget 的完整换算。